PROJECT DOCUMENTATION

PROJECT TITLE: BLOGNEST

TEAM ID: SWTID-2026-6499

TEAM SIZE: 4

TEAM LEADER: DEVADHARSHINI S – 24csdevadharshini.s108@gmail.com

TEAM MEMBERS:

- ZARINA FIRTHOUS S - 24cszarinafirthous107@gmail.com
- JAYASRI M - 24csjayasri.m109@gmail.com
- VEERA DHAYA DHARANI S - 24csveeradhayadharani110@gmail.com

ABSTRACT

BlogNest is a backend-based blog management application developed using Node.js, Express.js, and MongoDB. The project focuses on creating and testing RESTful APIs for a blog system. The backend is developed and managed using Visual Studio Code, while Thunder Client is used to test API requests and responses. The project demonstrates basic GET and POST API operations and establishes a connection with MongoDB.

INTRODUCTION

BlogNest is designed as a backend application for handling blog-related API requests. The system uses Node.js as the runtime environment and Express.js to create the web server and API routes. MongoDB is used as the database technology, and Mongoose is used to establish the database connection. Thunder Client provides a convenient environment for testing the APIs without requiring a frontend application.

PROBLEM STATEMENT

A blog management system requires a backend that can receive client requests, process data, communicate with a database, and return appropriate responses. BlogNest addresses these basic backend requirements by implementing a Node.js and Express.js server with MongoDB connectivity and API testing through Thunder Client.

OBJECTIVES

• To develop a backend application using Node.js and Express.js.

• To establish a connection between the application and MongoDB.

• To implement GET and POST API endpoints.

• To test API requests and responses using Thunder Client.

• To understand the basic workflow of a REST API-based backend application.

TECHNOLOGIES USED

• Node.js – JavaScript runtime environment used to execute the backend.

• Express.js – Web framework used to create the server and API routes.

• MongoDB – NoSQL database technology used for database connectivity.

• Mongoose – ODM library used for MongoDB connection and database operations.

• Thunder Client – API testing tool used to send GET and POST requests.

• Visual Studio Code – Development environment used to create and manage the project.

SYSTEM REQUIREMENTS

Hardware Requirements:

• Computer/Laptop

• Minimum 4 GB RAM

• Internet connection for package installation and development support

Software Requirements:

• Node.js

• MongoDB

• Visual Studio Code

• Thunder Client extension

• Web browser

SYSTEM ARCHITECTURE

The BlogNest backend follows a simple client-server architecture. Thunder Client acts as the API client and sends requests to the Express.js server. The Node.js server processes the requests and communicates with MongoDB when database connectivity is required. The server then sends an appropriate HTTP response back to the client.

Flow:

Thunder Client → Express.js / Node.js Server → MongoDB

↓

HTTP Response

PROJECT STRUCTURE

A typical BlogNest backend structure is:

```
blognest/
├── config/
│   └── db.js
├── routes/
│   └── blogRoutes.js
├── server.js
├── package.json
└── .env
```

The exact folder structure may vary depending on the implementation, but the main backend components are the server, database connection, routes, package configuration, and environment variables.

BACKEND IMPLEMENTATION

The backend is developed using Node.js and Express.js. Express middleware is used to process JSON request data. The application runs on port 5000 unless another port is specified through the environment configuration.

The main server responsibilities are:

• Starting the BlogNest server.

• Processing incoming HTTP requests.

• Providing GET and POST API endpoints.

• Connecting the application to MongoDB.

• Returning JSON or text responses to the client.

MONGODB DATABASE CONNECTION

MongoDB is used as the database technology for the BlogNest project. Mongoose is used to connect the Node.js application to MongoDB.

Example database connection:

```
const mongoose = require("mongoose");

const connectDB = async () => {
   try {
      await mongoose.connect(process.env.MONGO_URI);
      console.log("MongoDB connected");
   } catch (error) {
      console.error("MongoDB connection failed:", error.message);
      process.exit(1);
   }
};
```

module.exports = connectDB;

The successful terminal message “MongoDB connected” confirms that the application has established the database connection.

GET API IMPLEMENTATION

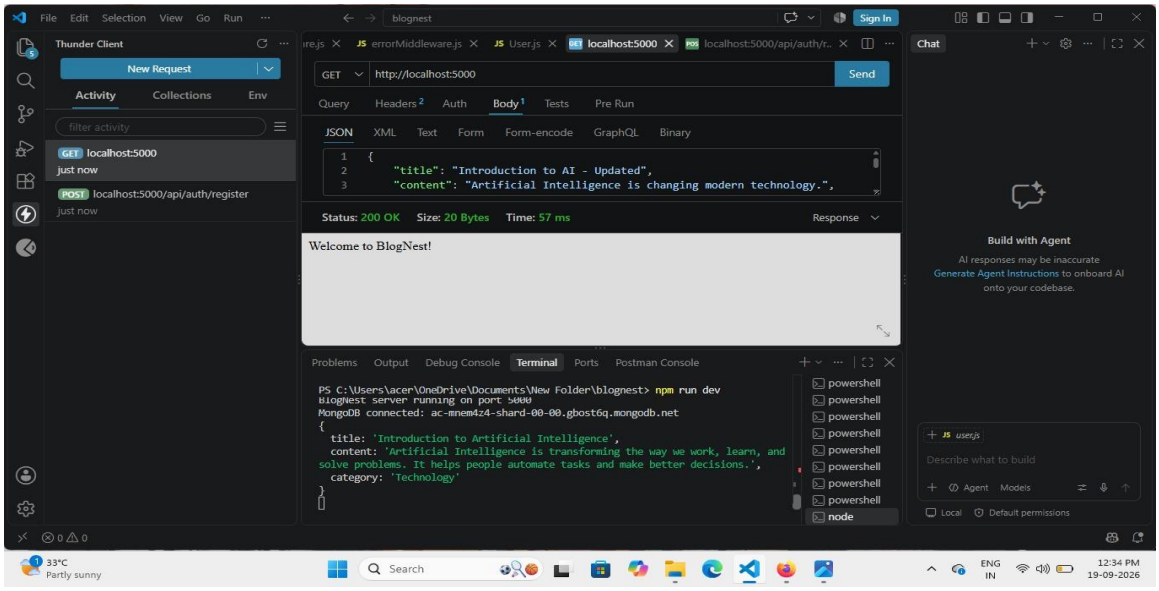
The GET API is used to check the availability of the BlogNest server.

Endpoint:

GET http://localhost:5000/

Expected response:

Welcome to BlogNest!



The successful request returns HTTP status code 200 OK. This confirms that the server is running and able to receive and respond to client requests.

Suggested Screenshot Caption:

Figure 1 – Testing the BlogNest GET API using Thunder Client

POST API IMPLEMENTATION

The POST API is used to send blog information to the BlogNest backend.

Endpoint:

POST http://localhost:5000/api/blogs

Example request data:

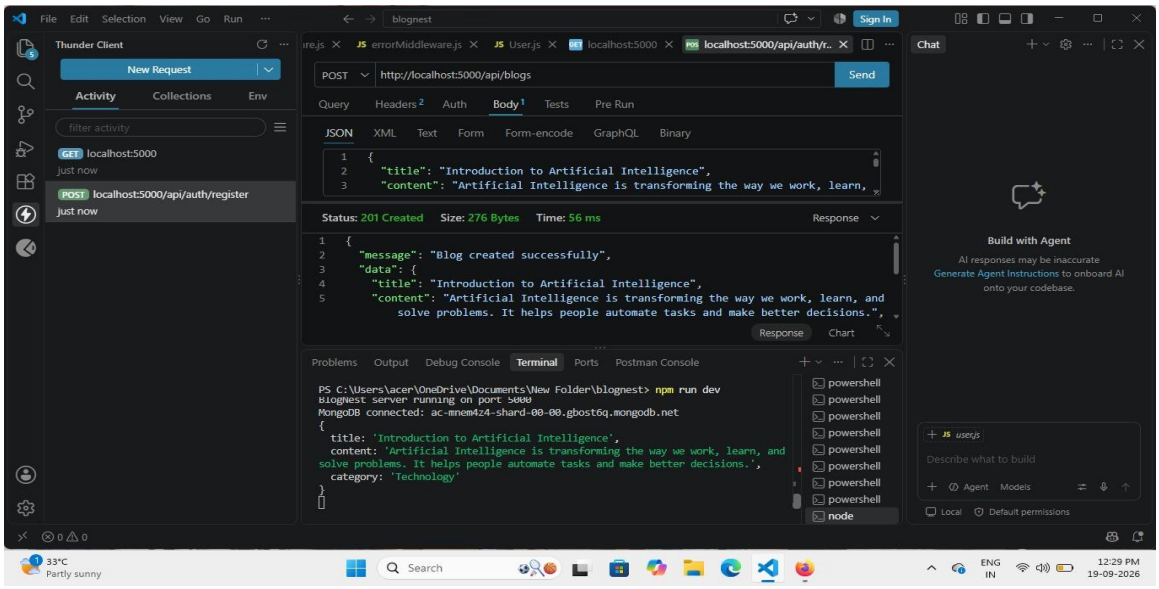

```
{
 "title": "My First Blog",
 "content": "This is my first blog post.",
 "category": "Technology"
}
```


The demonstrated API returns HTTP status code 201 Created with a response such as:


```
{
 "message": "Blog created successfully",
 "data": {
  "title": "My First Blog",
  "content": "This is my first blog post.",
```

```
  "category": "Technology"
 }
}
```

The current demonstrated POST implementation returns the submitted data in the response. It should not be described as permanently storing the blog in MongoDB unless database-save logic is implemented in the project.



Suggested Screenshot Caption:

Figure 2 – Testing the BlogNest POST API using Thunder Client

API TESTING USING THUNDER CLIENT

Thunder Client is used to test the BlogNest APIs directly from Visual Studio Code. It allows the developer to select the HTTP method, enter the API URL, provide request data, send the request, and inspect the response status and body.

Testing performed:

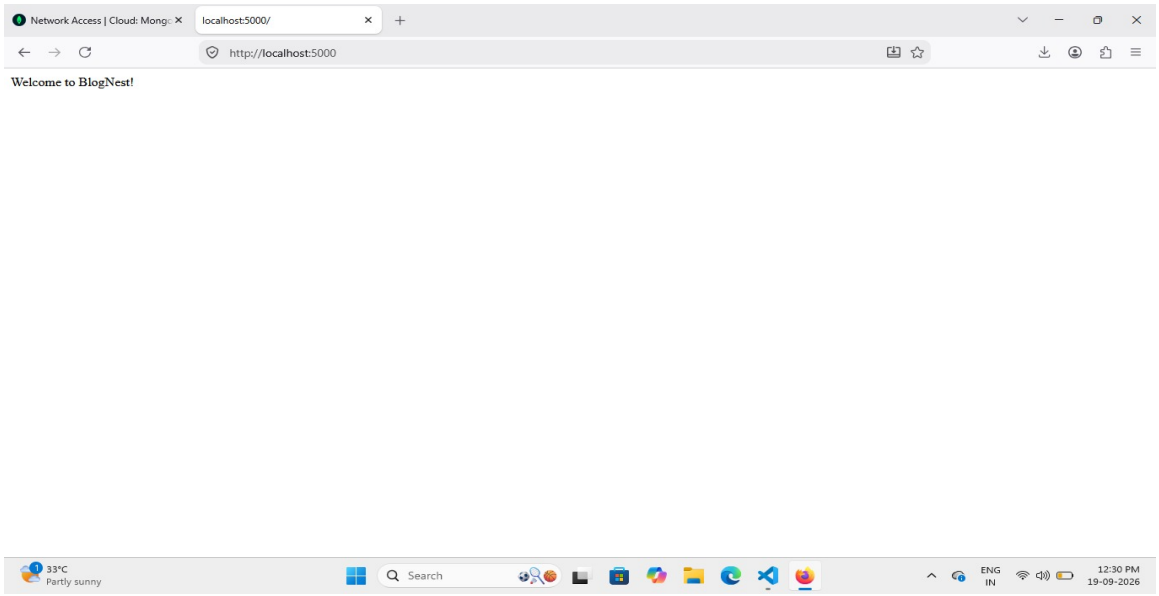
1. GET request to the root endpoint.
2. POST request to the blog endpoint.
3. Verification of HTTP status codes.
4. Verification of response messages and submitted data.

Suggested Screenshot Caption:

Figure 3 – Successful API response in Thunder Client

OUTPUT

The BlogNest backend was successfully executed on port 5000. The GET API produced a 200 OK response with the welcome message. The POST API produced a 201 Created response with the success message and submitted blog information. The terminal also displayed the server-running message and MongoDB connection status.



Suggested Screenshot Captions:

Figure 3 - BlogNest Application Running on Localhost

ADVANTAGES

• Simple and easy-to-understand backend architecture.

• Uses Node.js and Express.js for efficient API development.

• MongoDB provides a flexible NoSQL database environment.

• Thunder Client makes API testing simple.

• The project can be extended with additional blog management features.

DISADVANTAGES

• The current project provides only basic GET and POST API functionality.

• No user authentication or authorization is implemented.

• Error handling and input validation are limited.

• The current demonstrated POST API does not permanently store blog data unless database-save logic is added.

• There is no frontend interface for direct user interaction.

• Advanced features such as search, pagination, and image upload are not currently included.

FUTURE ENHANCEMENT

The BlogNest backend can be extended with:

• Complete MongoDB CRUD operations.

• Blog update and delete APIs.

• User registration and login.

• Authentication and authorization.

• Blog search and filtering.

• Image upload support.

• Pagination and sorting.

• Input validation and improved error handling.

CONCLUSION

BlogNest demonstrates the development of a basic backend blog management application using Node.js, Express.js, and MongoDB. The project successfully establishes a backend server, connects to MongoDB, and provides GET and POST API endpoints. Thunder Client is used to verify the API responses. The project provides a foundation for developing a more complete blog management system in the future.

REFERENCE

1. Node.js Documentation – Official documentation for Node.js.

2. Express.js Documentation – Official documentation for Express.js.

3. MongoDB Documentation – Official documentation for MongoDB.

4. Mongoose Documentation – Official documentation for MongoDB object modeling.

5. Thunder Client Documentation – API testing tool documentation.

6. Visual Studio Code Documentation – Development environment documentation.

DEMO VIDEO LINK

https://drive.google.com/file/d/1iDkIdqZfzpaWUDW8F-DfV_3GWvDbKMTH/view?usp=drivesdk